# SESSION 9 — Mini NumPy Case Study

## Objective of This Session

In this session we will connect everything learned in previous sessions and apply it to a small real-world dataset.

So far we have learned:

Array Creation
Mathematical Operations
Statistical Functions
Indexing and Slicing
Array Manipulation
Utility Functions
Handling Missing Values

Now we will apply these concepts to a simple dataset.

---

## Case Study: Store Sales Dataset

Assume we have data from a small store.

Each product has:

- Price
- Quantity sold

From this dataset we want to compute:

Total revenue
Average price
Products with highest sales
Clean missing values
Generate summary statistics

This is similar to real data preprocessing tasks in data science.

---

## Step 1 — Creating the Dataset

We create two arrays:

Prices of products
Quantity sold

```
import numpy as np

# Price of products
prices = np.array([100, 200, 150, 300, np.nan, 250])

# Quantity sold
quantity = np.array([10, 5, 8, 3, 7, 6])

print("Prices:", prices)
print("Quantity:", quantity)
```

```
Prices: [100. 200. 150. 300.  nan 250.]
Quantity: [10  5  8  3  7  6]
```

## Step 2 — Data Cleaning

Real-world datasets often contain missing values.

In our dataset one price value is missing (NaN).

We will:

1. Identify missing values
2. Replace missing values with average price

```
# Detect missing values
print("Missing values in prices:", np.isnan(prices))

# Calculate average price ignoring NaN
avg_price = np.nanmean(prices)

print("Average price:", avg_price)

# Replace NaN with average price
prices = np.where(np.isnan(prices), avg_price, prices)

print("Cleaned prices:", prices)
```

```
Missing values in prices: [False False False False  True False]
Average price: 200.0
Cleaned prices: [100. 200. 150. 300. 200. 250.]
```

## Step 3 — Perform Arithmetic Operations

Revenue = Price × Quantity

This uses element-wise multiplication.

```
revenue = prices * quantity
```

```
print("Revenue for each product:", revenue)
```

```
Revenue for each product: [1000. 1000. 1200.  900. 1400. 1500.]
```

## Step 4 — Generate Summary Statistics

Now we compute important statistics:

Total revenue
Average revenue
Maximum revenue
Minimum revenue

```
print("Total Revenue:", np.sum(revenue))

print("Average Revenue:", np.mean(revenue))

print("Maximum Revenue:", np.max(revenue))

print("Minimum Revenue:", np.min(revenue))
```

```
Total Revenue: 7000.0
Average Revenue: 1166.6666666666667
Maximum Revenue: 1500.0
Minimum Revenue: 900.0
```

## Step 5 — Identify Best Selling Product

We want to find which product generated the highest revenue.

For this we use:

np.argmax()

This returns the index of the maximum value.

```
best_product_index = np.argmax(revenue)

print("Best selling product index:", best_product_index)

print("Revenue generated:", revenue[best_product_index])
```

```
Best selling product index: 5
Revenue generated: 1500.0
```

## Step 6 — Sorting Products by Revenue

Sorting helps us rank products.

We use:

np.argsort()

```
sorted_indices = np.argsort(revenue)

print("Sorted product indices:", sorted_indices)

print("Revenue sorted:", revenue[sorted_indices])
```

```
Sorted product indices: [3 0 1 2 4 5]
Revenue sorted: [ 900. 1000. 1000. 1200. 1400. 1500.]
```

## Step 7 — Identify High Revenue Products

Let us find products generating revenue greater than 1000.

We will use boolean indexing.

```
high_revenue_products = np.where(revenue > 1000)

print("Indices with revenue > 1000:", high_revenue_products)

print("Revenue values:", revenue[high_revenue_products])
```

```
Indices with revenue > 1000: (array([2, 4, 5]),)
Revenue values: [1200. 1400. 1500.]
```

## Step 8 — Final Dataset Summary

Now we create a small summary of the dataset.

Metrics:

Number of products
Total sales revenue
Average price
Average quantity

```
print("Number of products:", prices.size)

print("Total revenue:", np.sum(revenue))

print("Average price:", np.mean(prices))

print("Average quantity sold:", np.mean(quantity))
```

```
Number of products: 6
Total revenue: 7000.0
Average price: 200.0
Average quantity sold: 6.5
```

Start coding or generate with AI.